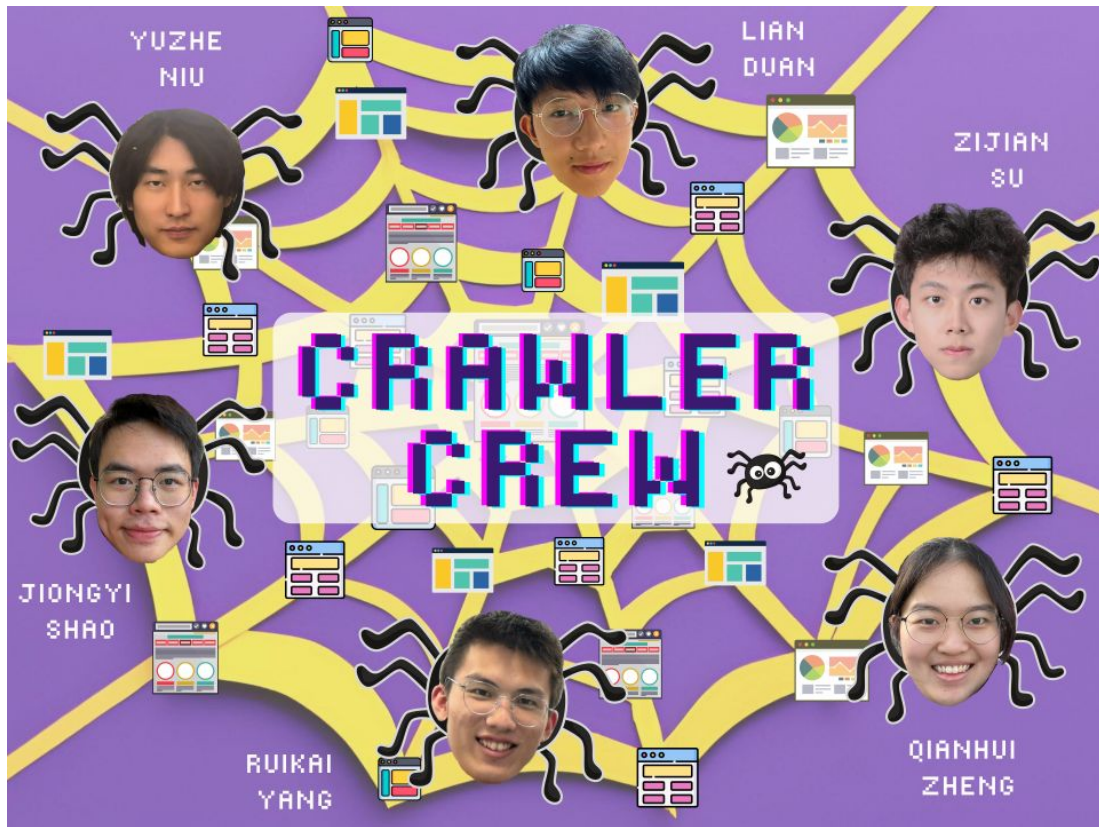


EECS 498 System Design: Small Scale Search Engine

Instructor: **Nicole Hamilton**

Presentor: **Crawler Crew**

Date: **04/22/2025**





Meet our group



Lian Duan
CS major
Junior



Qianhui Zheng
CS major
Junior



Jiongyi Shao
CS major
Junior

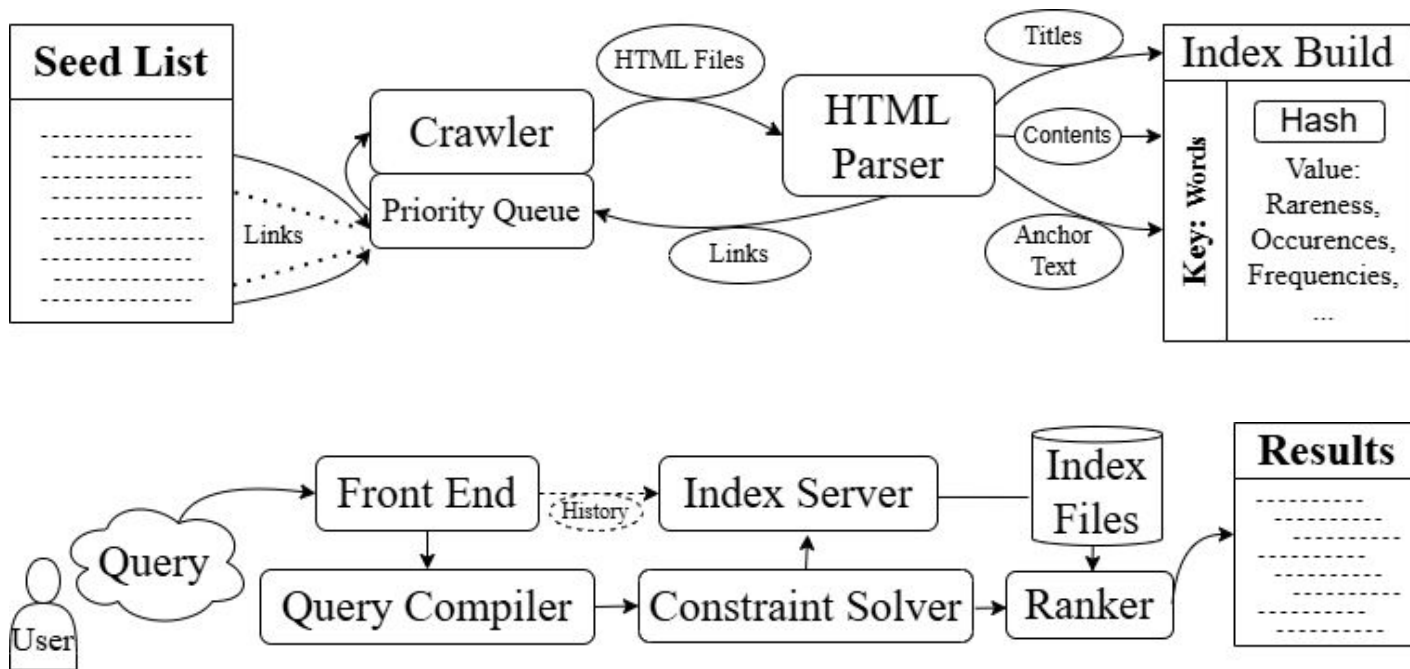


Yuzhe Niu
CE major
Junior

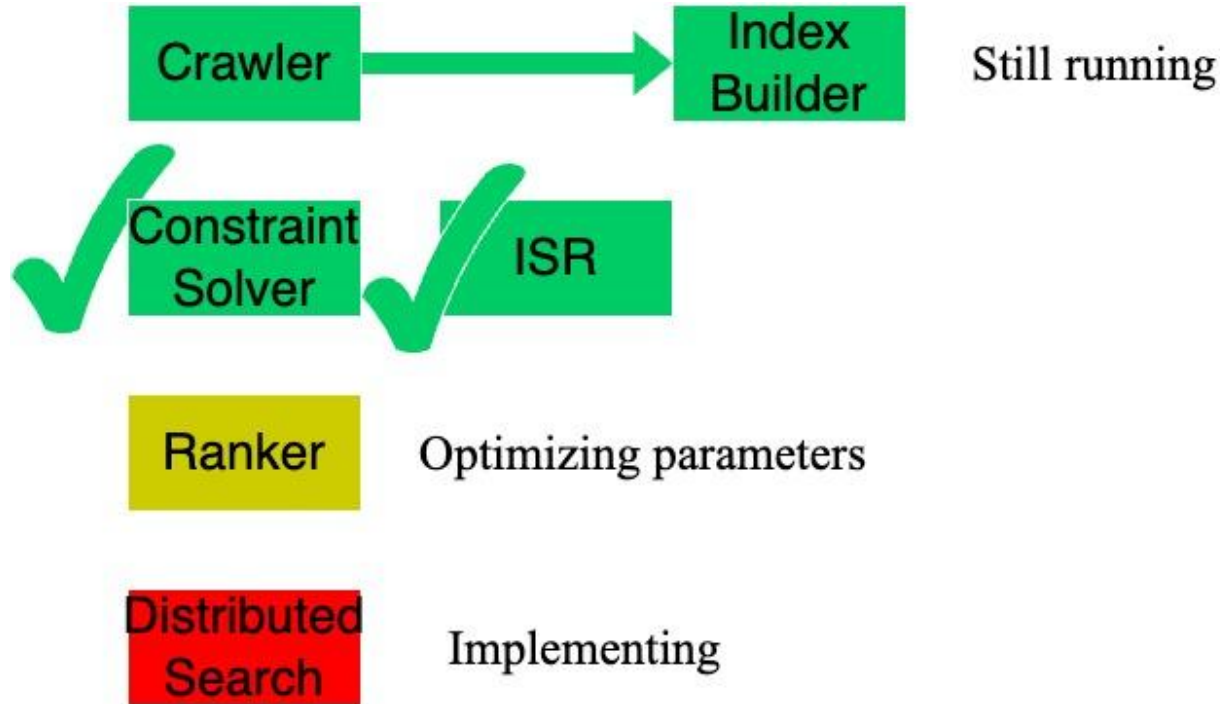


Ruikai Yang
CS major
Junior

System Overview



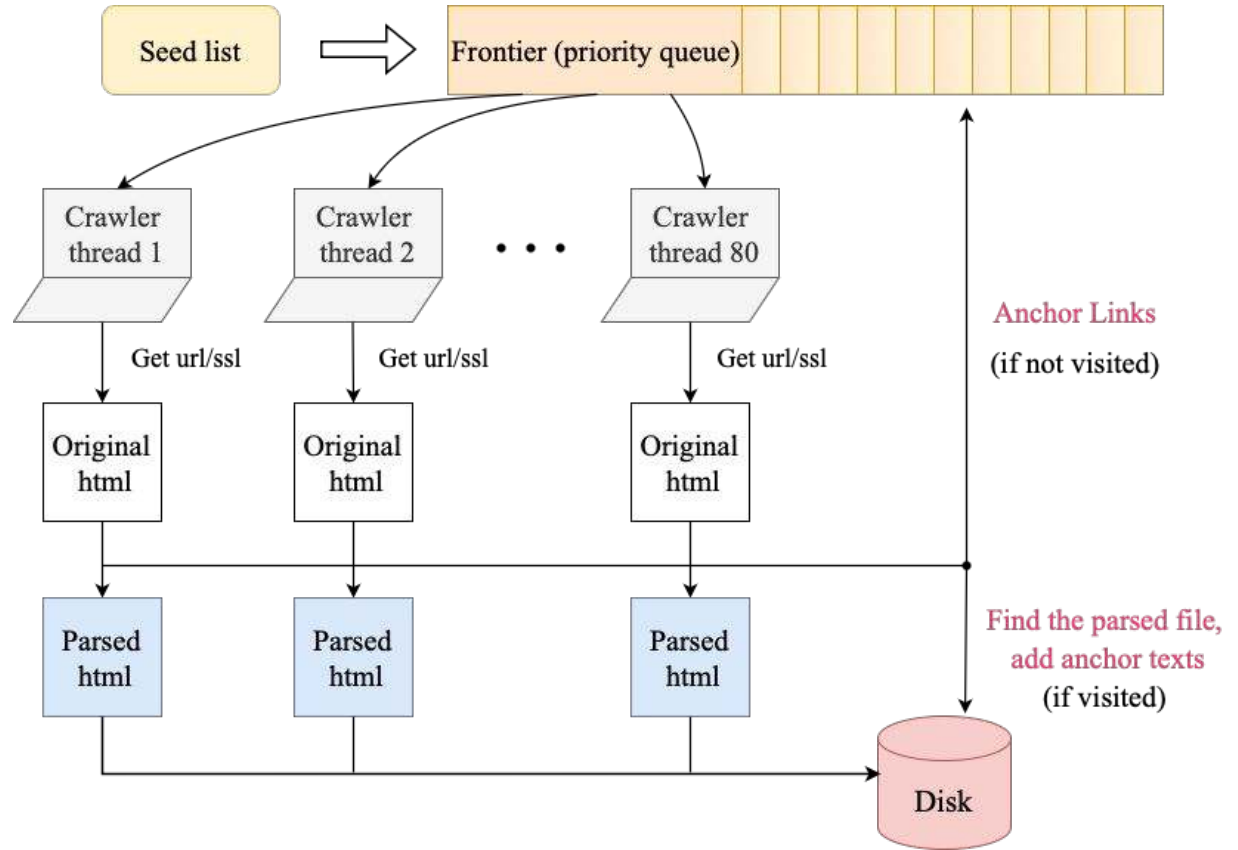
Progress



Crawler

Overview

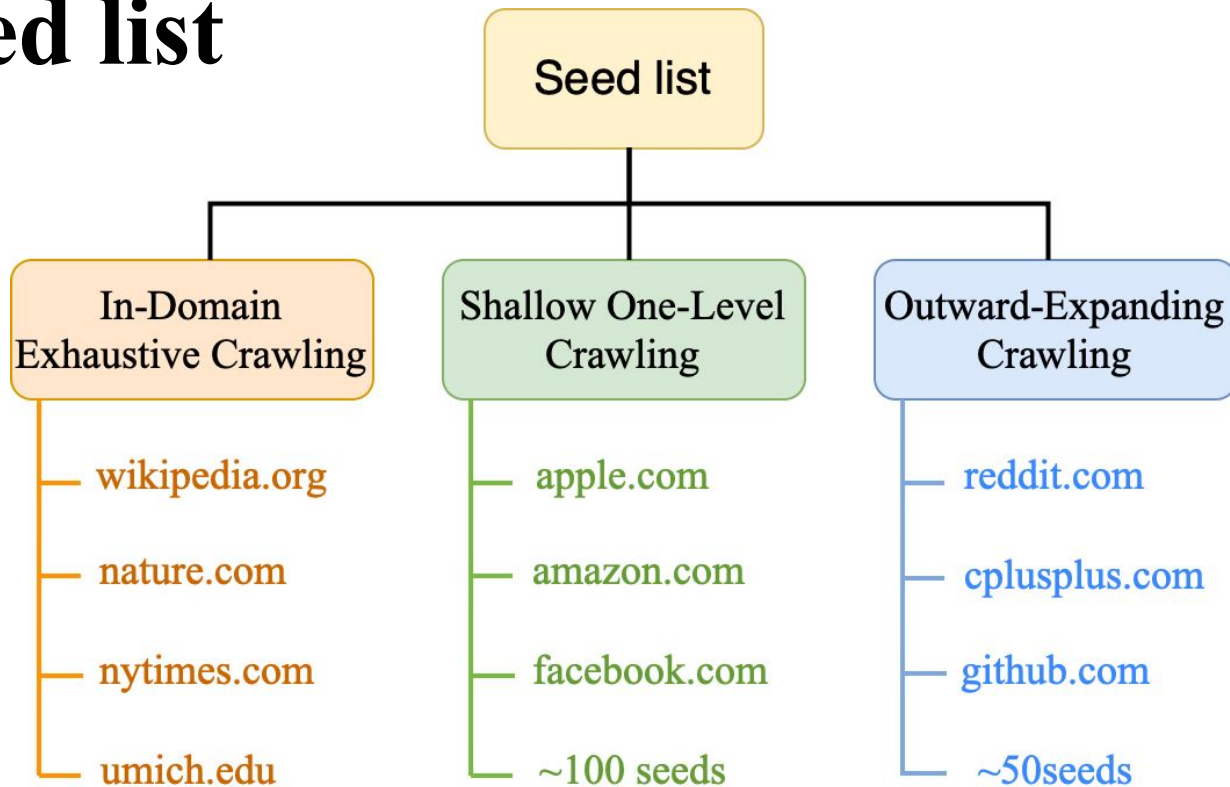
1. Seed list
2. Frontier
3. Crawler Threads
4. Html Parser
5. Visited Set



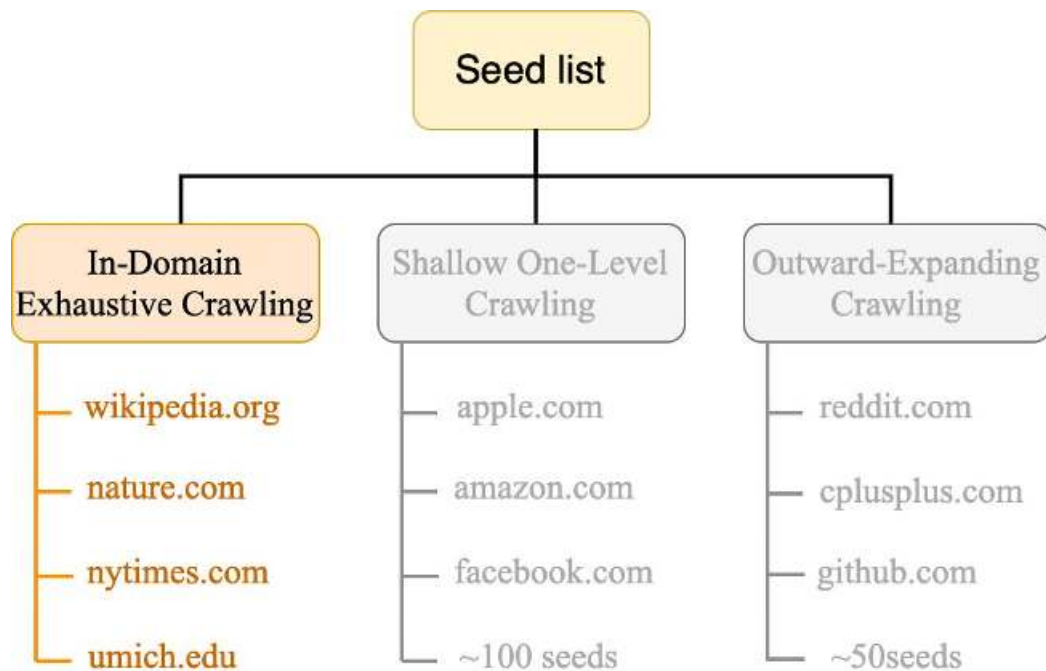
Core Design Decisions & Highlights

- a. Seed list design
- b. Frontier design - URL scoring strategy
- c. Anchor text
- d. Automatic Recrawling

a. Seed list



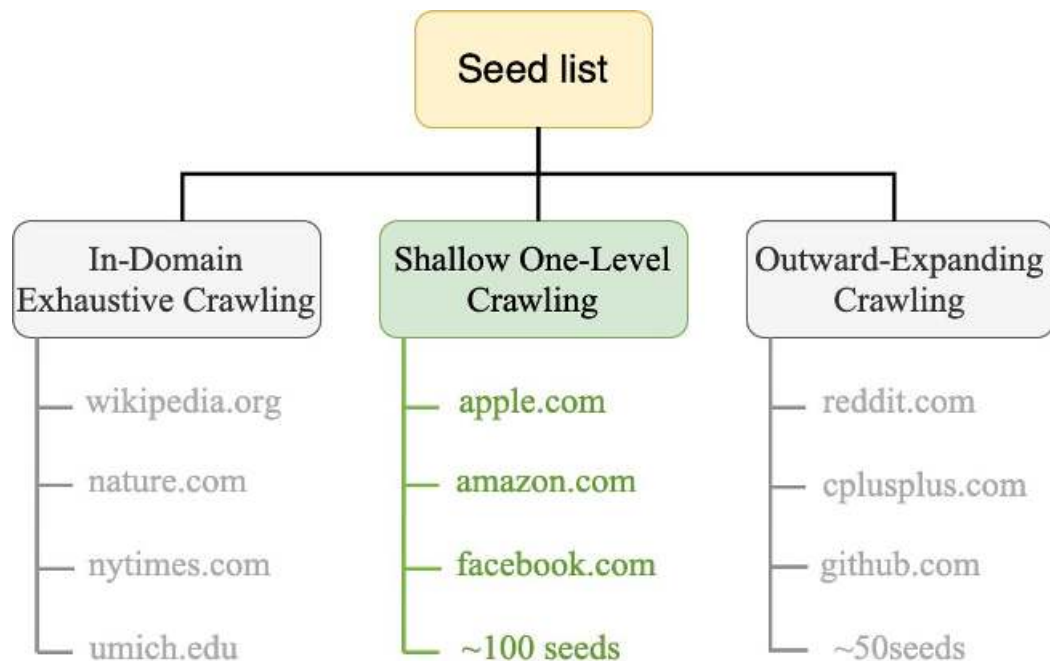
a. Seed list



1. In-Domain Exhaustive Crawling

- High-quality Domains
- Crawl within domains without following external links

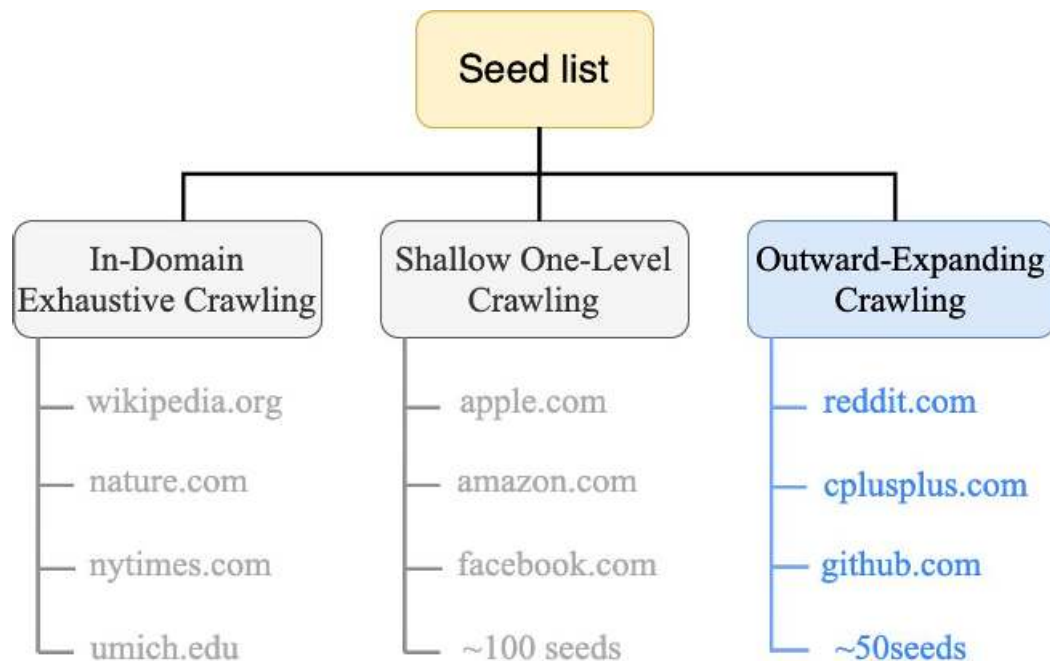
a. Seed list



2. Shallow One-Level Crawling

- Well-known company homepages
- Deeper contents involve protected or product-related information that **should not be crawled**

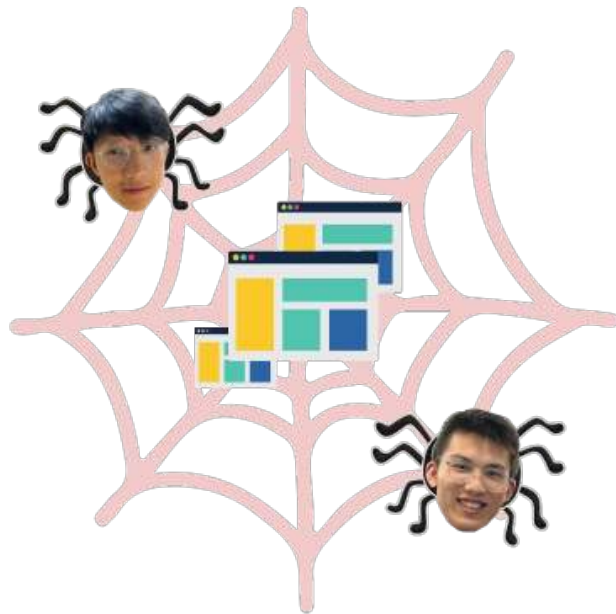
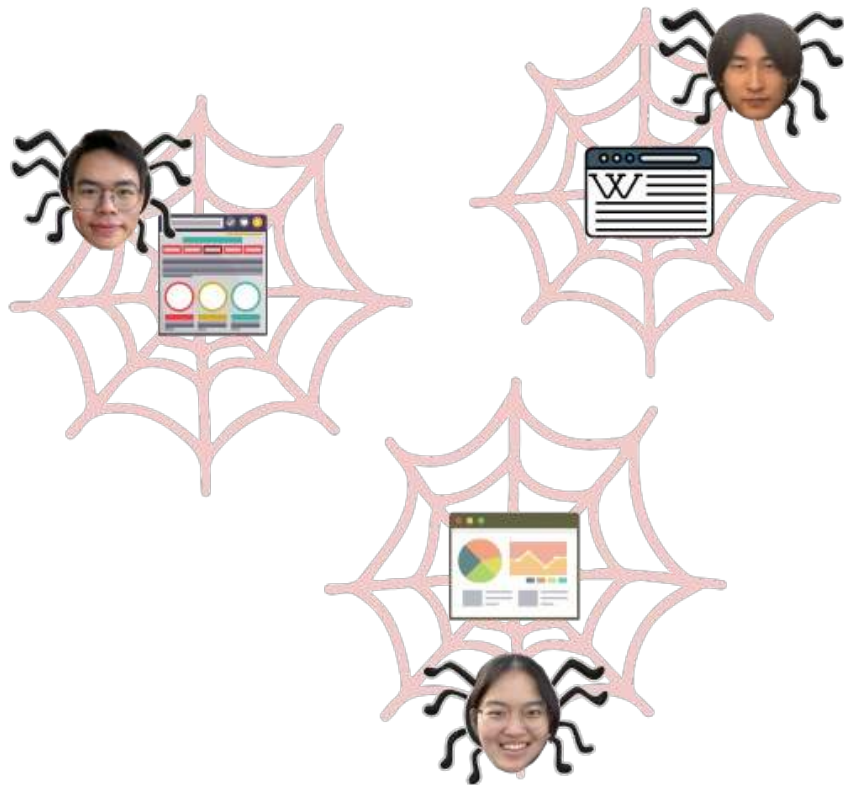
a. Seed list



3. Outward-Expanding Crawling

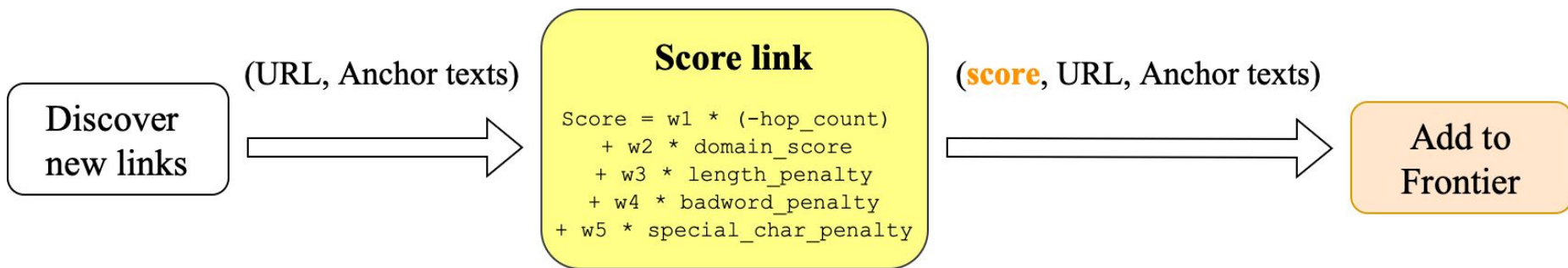
- Exploratory web discovery
- No depth limit

Customized Crawlers!



b. Frontier

- Implemented as a **priority queue**
- Each entry is a **tuple**: (score, URL, anchor_text)
- URLs are **scored at insertion** for efficient prioritization

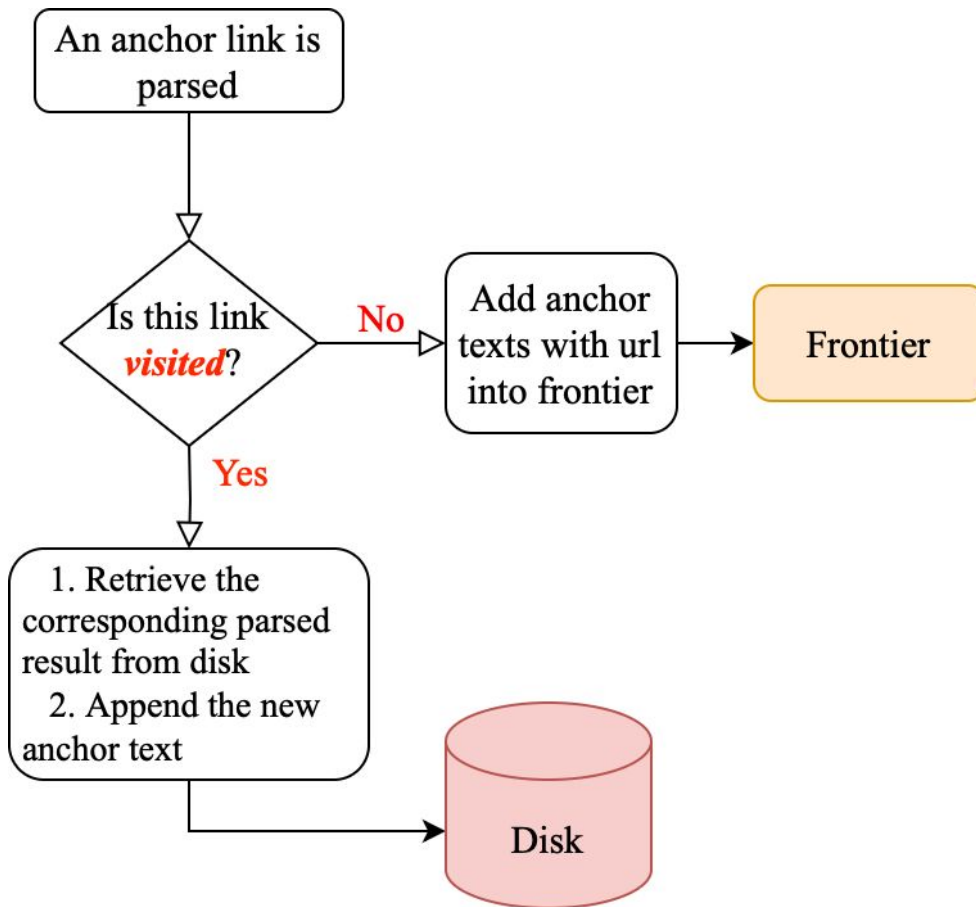


b. URL Scoring Strategy

Factor	Description	Weight
Hop Count	Distance from seed, encourages breadth-first crawling	★ Highest
Domain Authority	Prioritize trustworthy domains like .edu , .org , .gov	High
URL Length	Penalize long or deeply nested URLs (/count)	Medium
Blacklist Keywords	Strong negative weight for " login ", " register ", etc.	Medium
Special Characters	Penalize ? , # , or excessive numbers	Low

c. Anchor Texts

How we save anchor texts:



c. Anchor Texts

Why this matters?

- Mini summary of the page it points to
- Potential dataset for page rank

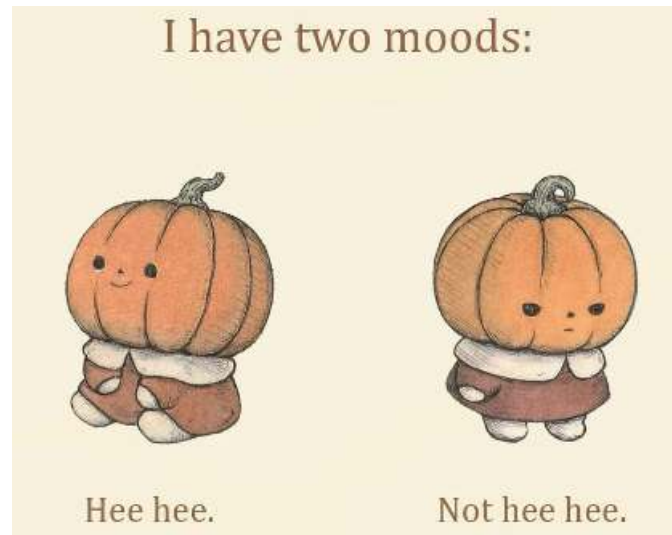
c. Anchor Texts

But... it comes with a cost!

- A huge increase in Disk I/O
- Slow down the crawler

Why this matters?

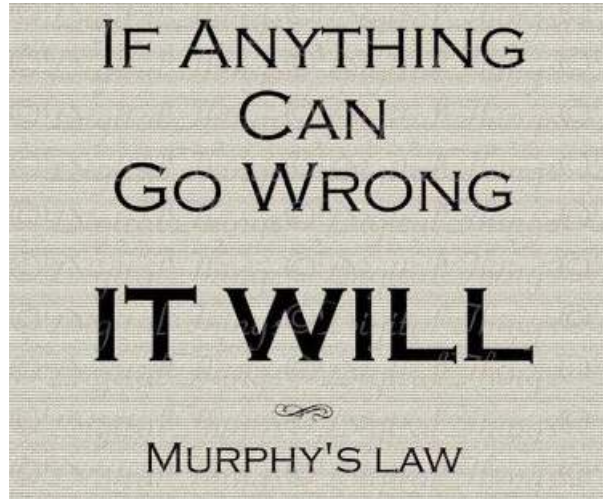
- Mini summary of the page it points to
- Potential dataset for page rank



d. Automatic Recrawling

What we initially thought:

"Our crawler is solid! It won't crash... right?"



d. Automatic Recovery

What we initially thought:

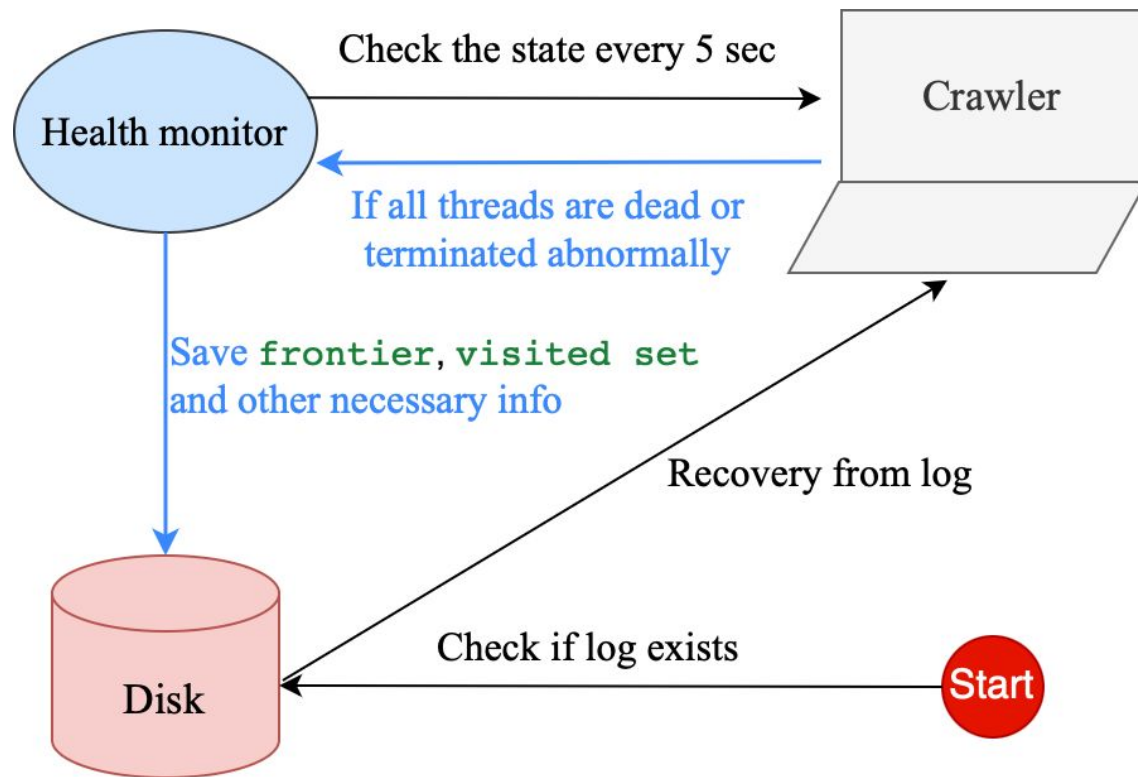
"Our crawler is solid! It won't crash... right?"

Even the cleanest crawler can die unexpectedly!

"Saving crawler state isn't optional - it's essential."

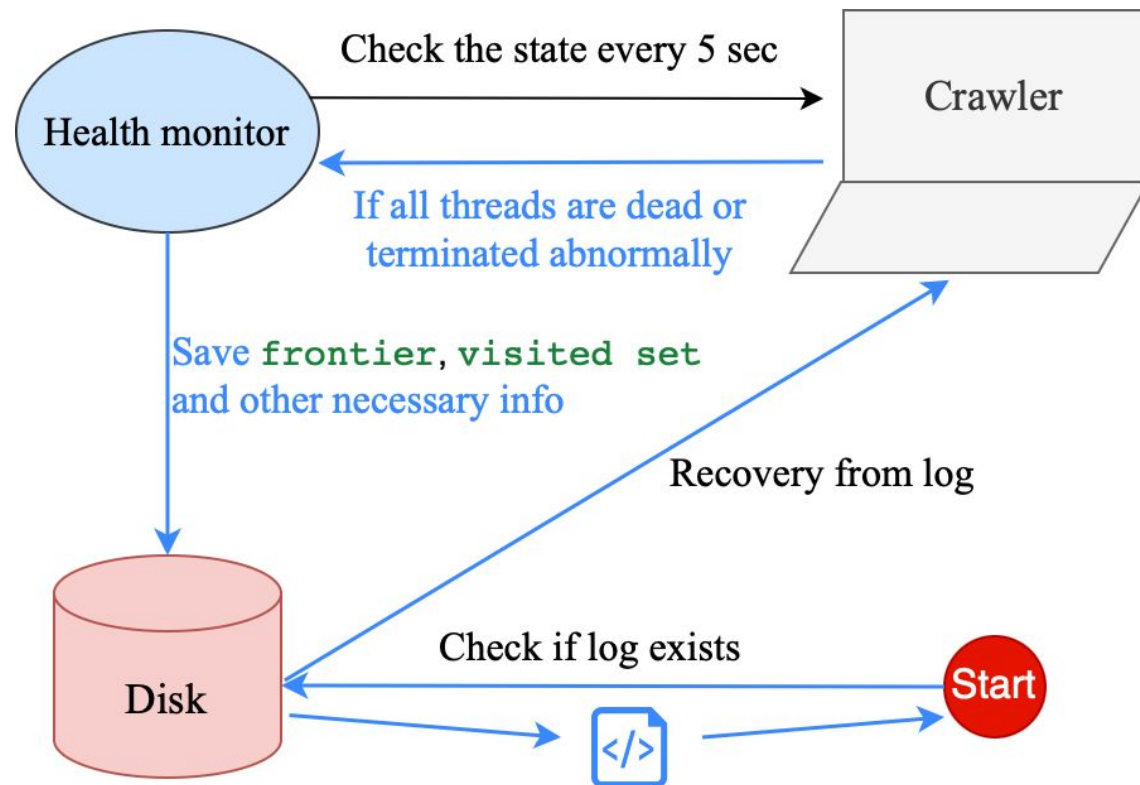
d. Automatic Recovery

What we did to fix:
automatic recovery
& **recrawling**



d. Automatic Recovery

What we did to fix:
automatic recovery
& **recrawling**



d. Automatic Recrawling

Nothing bad?

- Once the crawler starts... it's *really* hard to stop 🤖!!!
- We've had multiple laptops **freeze up** or get **overloaded** because the crawler kept reviving itself
- Let's just say: **resilient $\neq$ convenient**

Data about Crawler

Dimensions	Data
Seed list size in URLs	~150
Number of crawler threads per machine	80
Number of crawler machines	5
Estimated overall crawl speed (documents / day)	One-level crawler: 0.6M (no need for frontier) No depth limit crawler: 0.3M (need frontier, slow down progressively)
Number of documents crawled	~10M

Index

Overview

Stored as a HashFile ←

Dictionary	Posting List	Posting List	Posting List	Posting List	
------------	--------------	--------------	--------------	--------------	-------

One Posting List:

Header	Post	Post		Post	Seek Table
--------	------	------	-------	------	------------

Header:

Num of Occurences	Num of docs	Type	Total Size of this list	Hashvalue	Seek table offset	Seek table count
-------------------	-------------	------	-------------------------	-----------	-------------------	------------------

Post for word/doc end:

Delta	Word Attribute	Delta	Doc Attribute
-------	----------------	-------	---------------

Document Attribute:

Url	Title	Length of Doc	# Anchor Text	# unique words
-----	-------	---------------	---------------	----------------

Seek Table:

Seek Entry	Seek Entry		Seek Entry
------------	------------	-------	------------

One Seek Entry:

Word id	Byte Offset
---------	-------------

Index: Highlights

1. Index chunks are ordered by quality: First search good quality.
2. Simplification of words: Case Insensitive, delete punctuation, delete stopwords (570 words, saved as a .txt file)
3. Seek Table Design: 100 posts corresponds to 1 seek entry
4. Decoration
5. Saved in different machines

Good quality

Faster results



Highlights

Good quality

- a. Index chunks are ordered by quality: *First search good quality.*
- b. Simplification of words
- c. Decoration

Faster Results

- d. Seek Table Design:
100 posts corresponds to 1 seek entry
- e. Saved on different machines

b. Simplification of Words

example:	VS	example	
place.	VS	place	Same method are used in constraint solver
Apple	VS	apple	

Stopwords: 570 stopwords (and, or, anyway, above, for, etc.)
Decrease the size of the index significantly
(50,000 pages: 3.2G -> 2.5G)

c. Decoration

#word → Title words

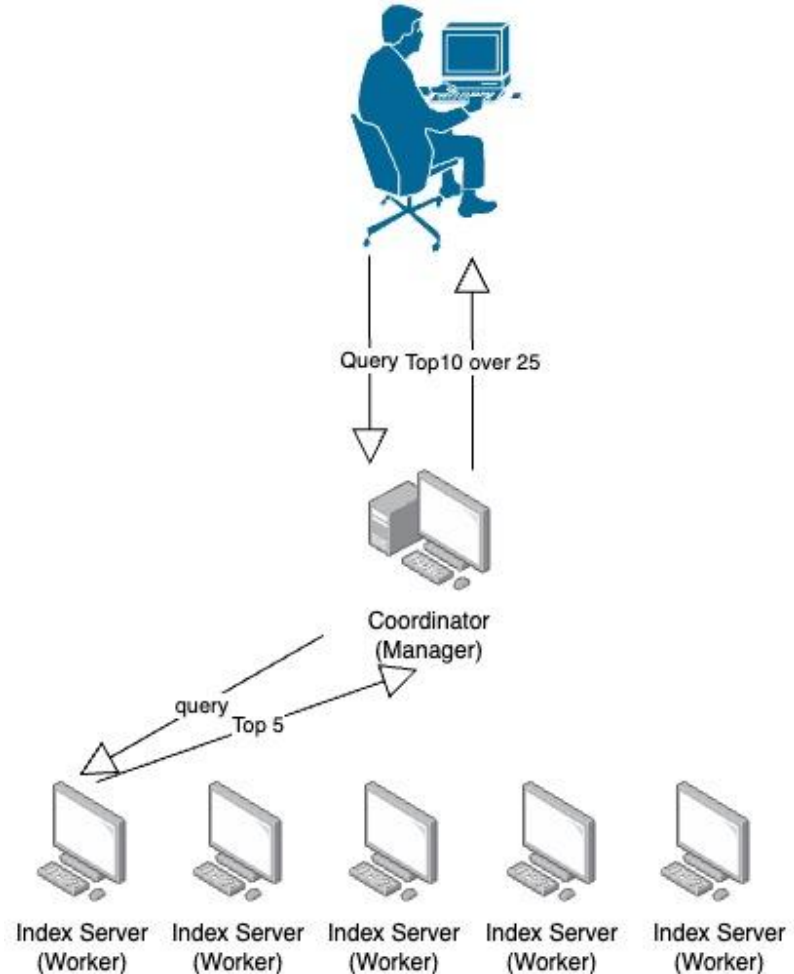
\$word → Anchor text

% → End of Document

In ranker part we know the position of the words. Thus we can score them differently.

Data about Index

1. Number of Document
Indexed: 10 million now
(ready to be 20 million)
2. Approximate Chunk Size:
300MB (about 5000 pages)
3. Number of machines the
index spread across: 5



What I learned

1. Need-driven coding.
2. Design wisely: Since need to change several times. Thus encapsulation is important.



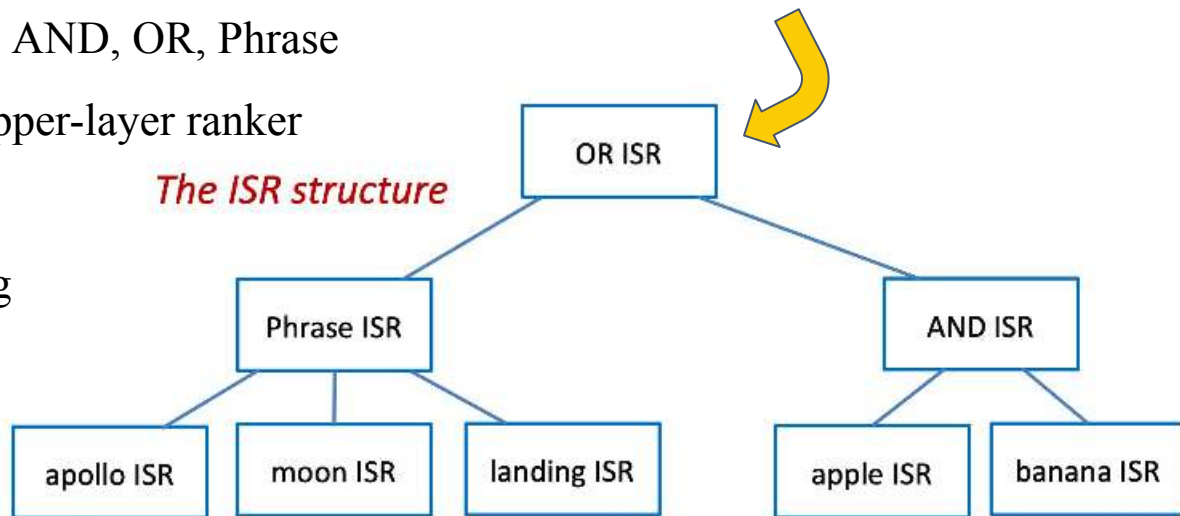
Query Compiler & ISR

Query Compiler & ISR

Design goal

"apollo moon landing" | (apple banana)

- a. Support nested expressions: AND, OR, Phrase
- b. Simplify interface for the upper-layer ranker
- c. Optimize for performance:
 - Efficient document skipping
 - Fast seek
 - Reduce redundant parsing



Query Compiler & ISR

Query Composition & Semantic Power

```
<Query> ::= <And> { '|' <And> }  
<And> ::= <Part> { <Part> }  
<Part> ::= '(' <Query> ')' | <Phrase> | ['$ ', '#']word  
<Phrase> ::= " word { word } "
```

Apple macos | Microsoft windows



OR(AND(Apple, macos) , AND(Microsoft, windows))

("Deep learning" | AI) courses umich



AND(OR(Phrase(deep, learning) , AI), courses, umich)

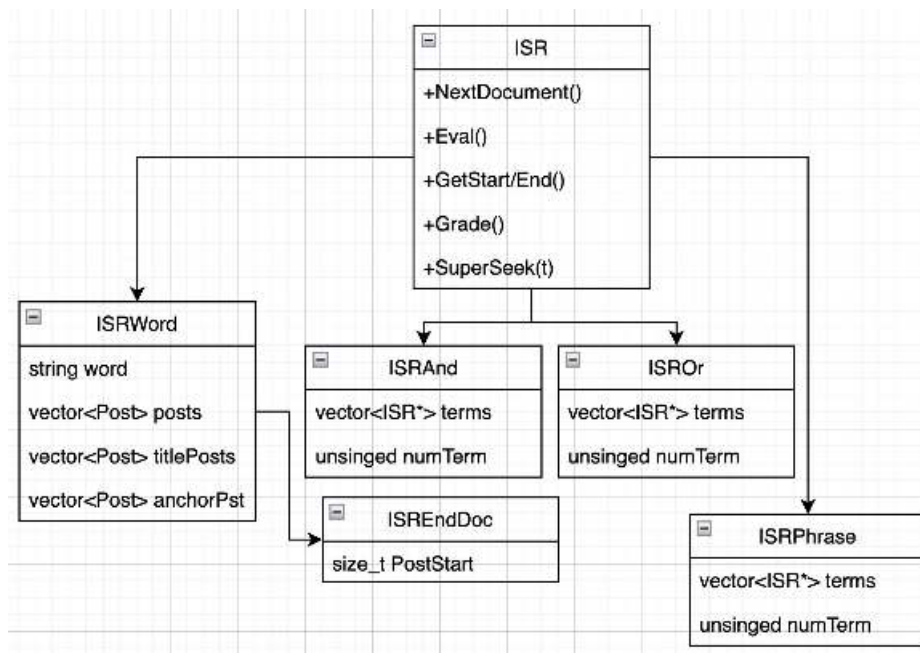
Query Compiler & ISR

Class Structure Overview

Inheritance Highlights

Deep inheritance structure enables complex query composition

Each subclass implements only its specific logic, but automatically integrates with composite structures



Query Compiler & ISR

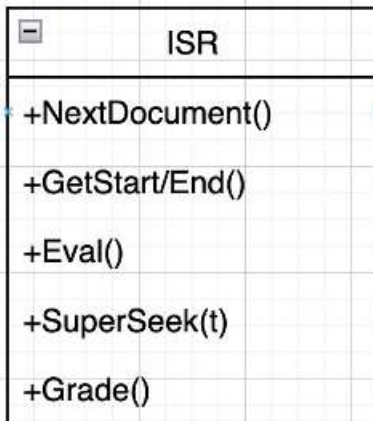
Interface Abstraction & Unification

Benefits of this design

Ranker decoupled from internal implementation details

Recursive composition is easy to manage and extend

Enhances maintainability and modularity



move to the next matching document

retrieve term span within the document

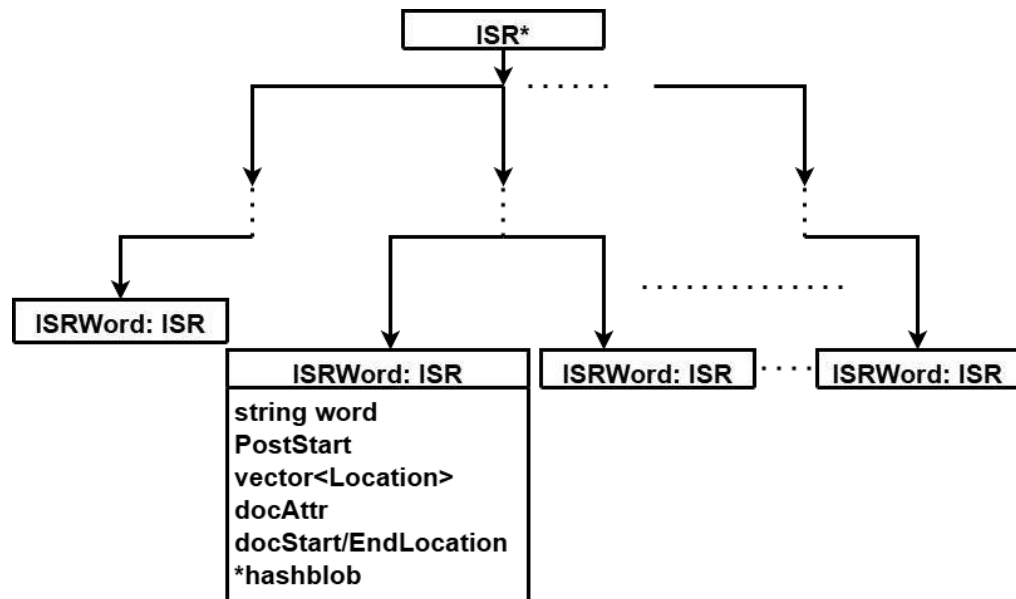
prepare data for scoring

skip to a target location efficiently

return the relevant score to current document

Query Compiler & ISR

Implementation Details: Basic Data Structures

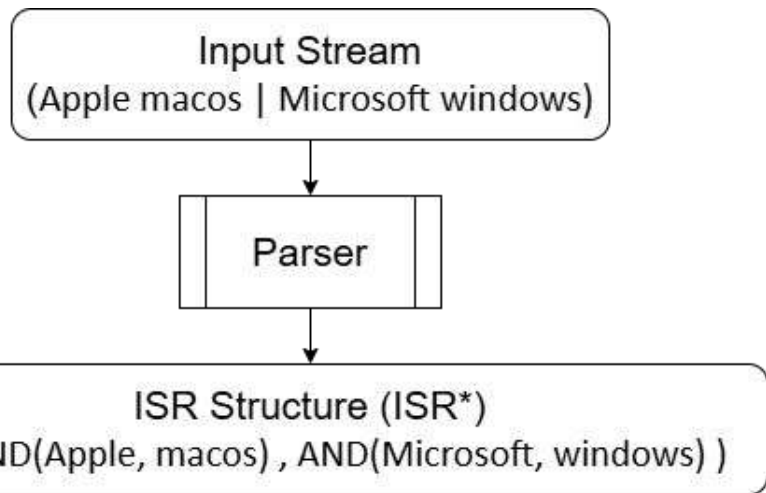


Inside **ISRWord**

1. **DocStart/End**: Set to 0 at the start
2. **PostStart**: Absolute position of the first word in the current doc
3. **vector<Location>**: vector(s) of absolute position of a word within the current doc
4. ***hashblob**: which Index we're in

Query Compiler & ISR

What happens when a query is being searched



```
for (All index chunks){  
  while(true){  
    size_t enddoc= isr->NextDocument();  
    isr -> UpdateDoc( enddoc );  
    isr -> Eval();  
    isr->Grade();  
  }  
}
```

Query Compiler & ISR

UpdateDoc(Location t) & Eval() & NextDocument()

NextDoc:

Always returns a **location** of the end of doc

ISRAnd/Phrase: Only returns the end of doc **location** where it contains all its children ISRs

Improve efficiency for ranking

UpdateDoc:

1. Always pass in a **location** of the end of doc
2. Update **docStart/EndLocation** and **docAttr**

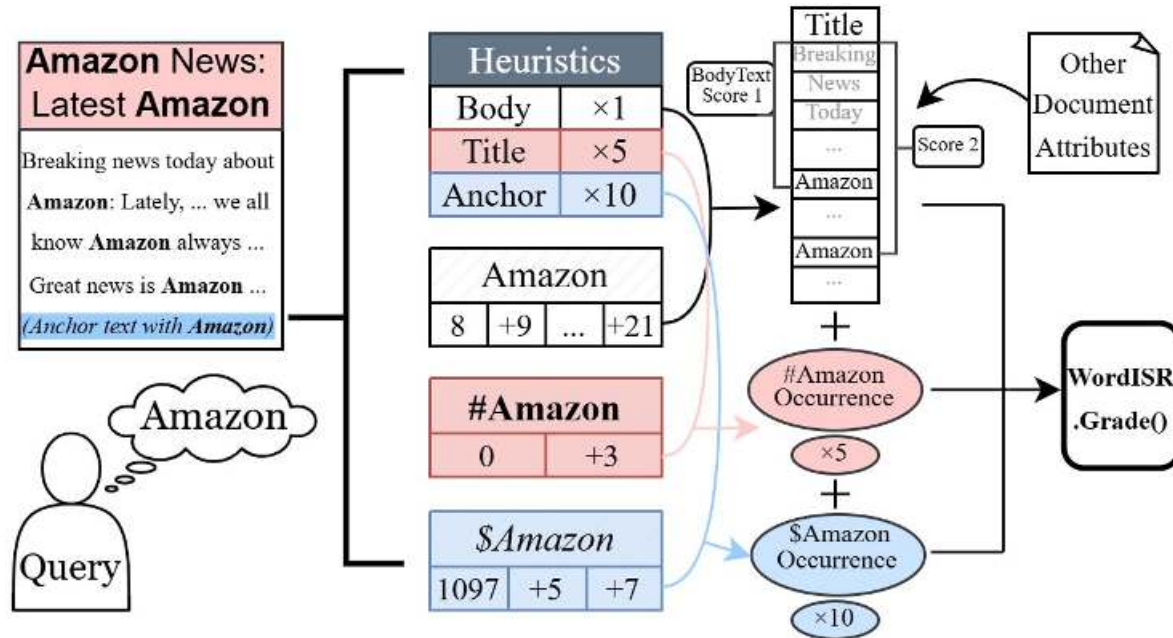
Eval:

Update the **vector of positions**

Ranker

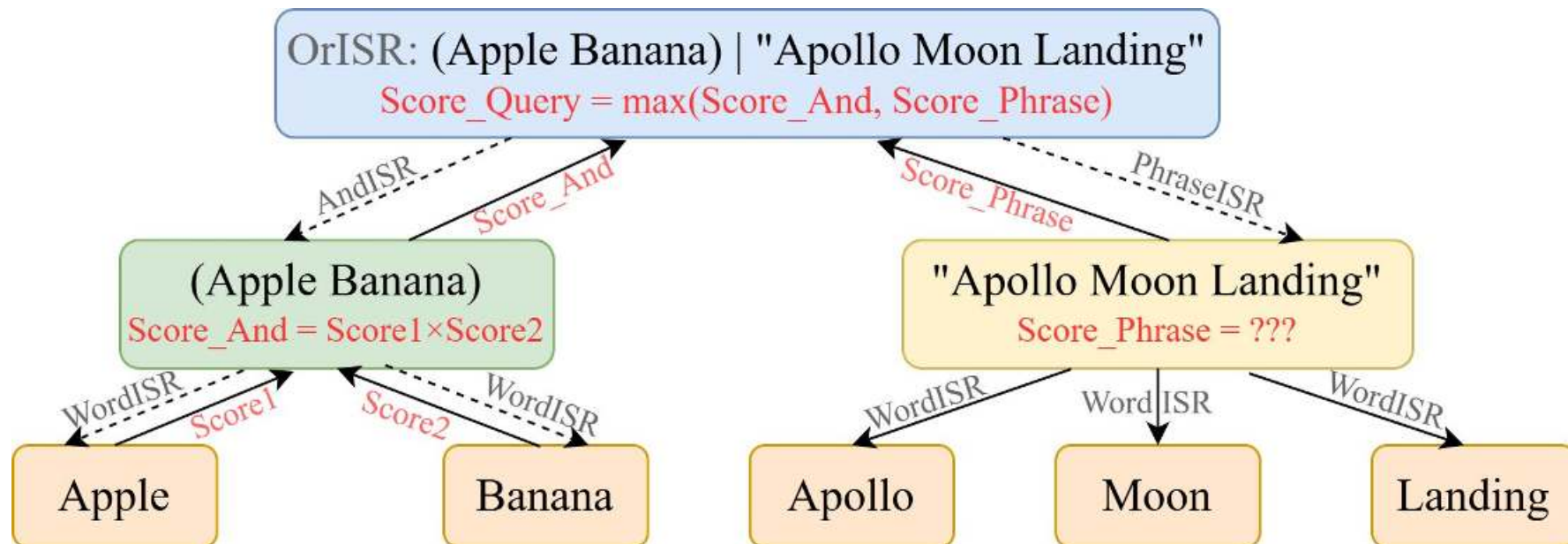
Ranker

For any document, Grading starts from a single WordISR ...



Ranker

Now that we know how to grade a single word ...



Ranker: How to Grade a Phrase?

Wiki Topic:
Quick Brown Fox

A quick brown fox phrase at the very top of the document,
...
A close quick and brown fox near the title here,
...
Another quick brown fox found as phrase at the bottom of this page.

Anchor Text: Quick Brown Quick Brown Fox



Ranker: How to Grade a Phrase?

Wiki Topic:
Quick Brown Fox

A quick brown fox phrase at the
very top of the document,

...

A close quick and brown fox near
the title here,

...

Another quick brown fox found as
phrase at the bottom of this page.

*Anchor Text: Quick Brown Quick
Brown Fox*



Ranker: How to Grade a Phrase?

Wiki Topic: Quick Brown Fox

A quick **brown fox** phrase at the
very top of the document,

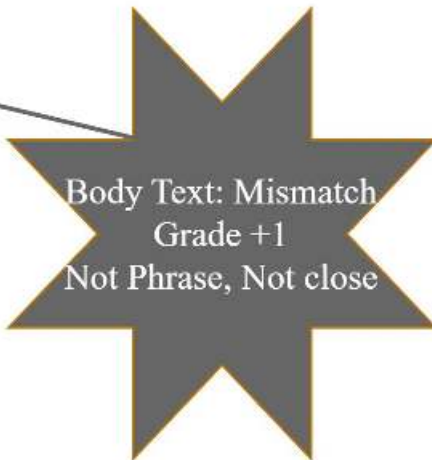
...

A close **quick** and brown fox near
the title here,

...

Another quick brown fox found as
phrase at the bottom of this page.

*Anchor Text: Quick Brown Quick
Brown Fox*



Body Text: Mismatch
Grade +1
Not Phrase, Not close

Ranker: How to Grade a Phrase?

Wiki Topic: Quick Brown Fox

A quick brown fox phrase at the very top of the document,

...

A close quick and brown fox near the title here,

...

Another quick brown fox found as phrase at the bottom of this page.

*Anchor Text: Quick Brown Quick
Brown Fox*

Body Text: Mismatch
Grade +1
Not Phrase, Not close

Ranker: How to Grade a Phrase?

Wiki Topic: Quick Brown Fox

A quick brown fox phrase at the
very top of the document,

...

A close quick and brown fox near
the title here,

...

Another quick brown fox found as
phrase at the bottom of this page.

*Anchor Text: Quick Brown Quick
Brown Fox*

Body Text: Close
Grade +30
Near the title!

Ranker: How to Grade a Phrase?

Wiki Topic:
Quick Brown Fox

A quick brown fox phrase at the
very top of the document,

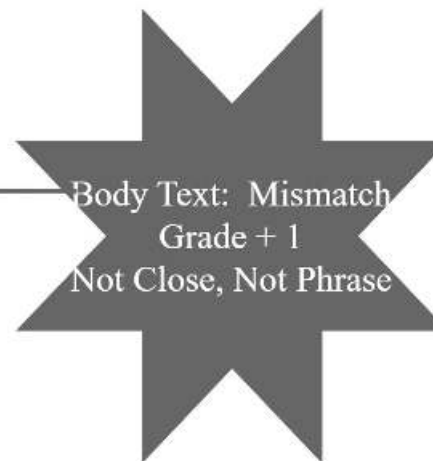
...

A close quick and **brown** **fox** near
the title here,

...

Another **quick** brown fox found as
phrase at the bottom of this page.

*Anchor Text: Quick Brown Quick
Brown Fox*



Ranker: How to Grade a Phrase?

Wiki Topic: Quick Brown Fox

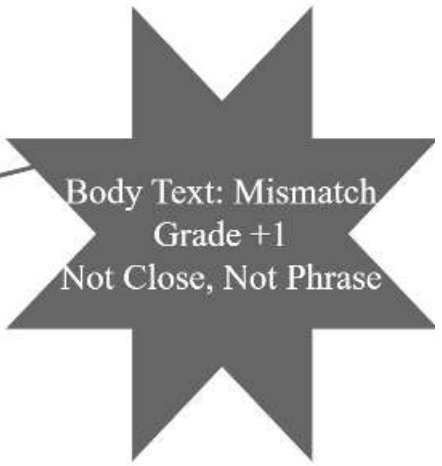
A quick brown fox phrase at the
very top of the document,

...

A close quick and brown **fox** near
the title here,

Another **quick** **brown** fox found as
phrase at the bottom of this page.

*Anchor Text: Quick Brown Quick
Brown Fox*



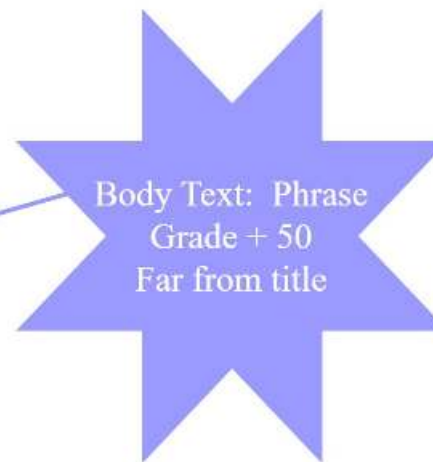
Body Text: Mismatch
Grade +1
Not Close, Not Phrase

Ranker: How to Grade a Phrase?

Wiki Topic:
Quick Brown Fox

A quick brown fox phrase at the
very top of the document,
...
A close quick and brown fox near
the title here,
...
Another quick brown fox found as
phrase at the bottom of this page.

*Anchor Text: Quick Brown Quick
Brown Fox*



Ranker: How to Grade a Phrase?

Wiki Topic:
Quick Brown Fox

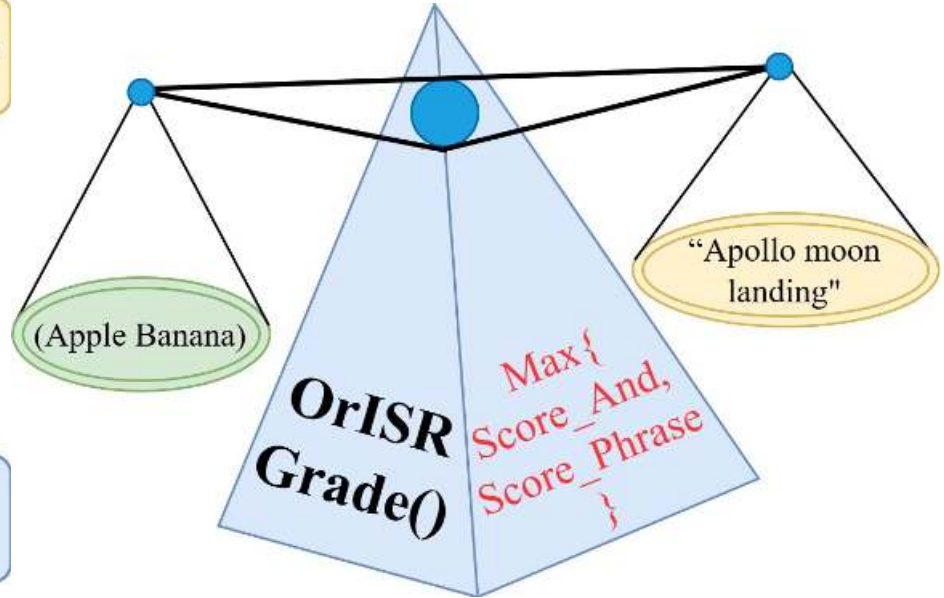
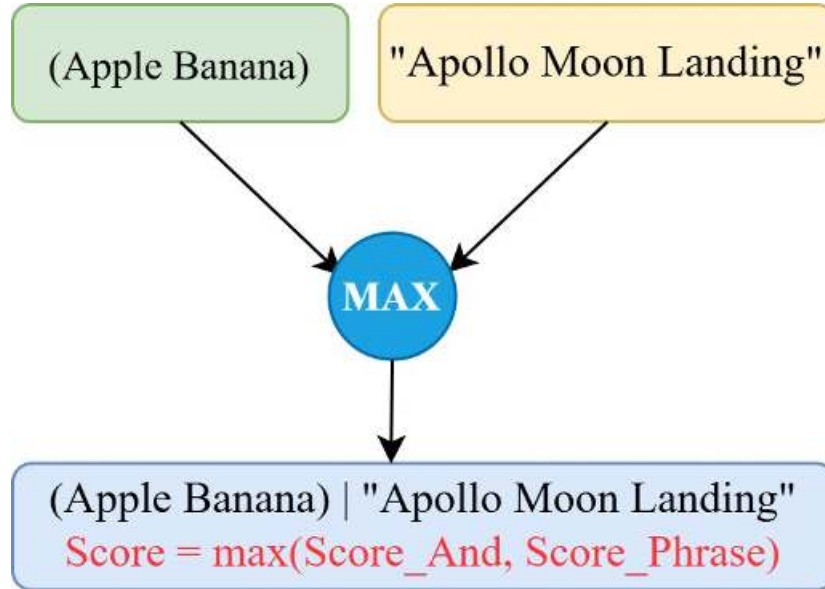
A quick brown fox phrase at the
very top of the document,
...
A close quick and brown fox near
the title here,
...
Another quick brown fox found as
phrase at the bottom of this page.

Anchor Text: **Quick Brown Quick
Brown Fox**

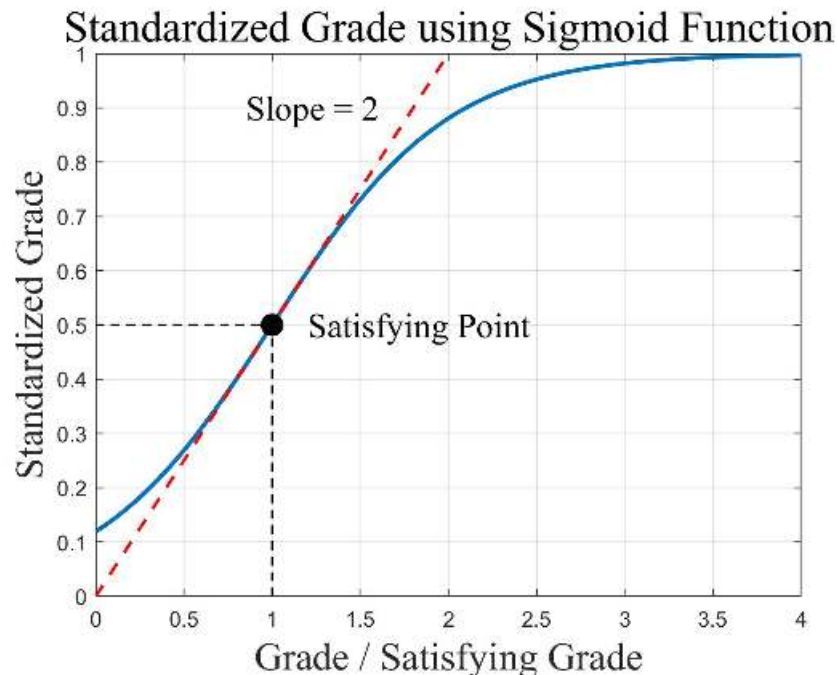
Anchor Text: 5 match
Grade + 5×10
We don't care
close or phrase here

Ranker

Wait, have we forgotten anything? What if one is always higher?



Ranker: We need Normalization



Satisfying Grade	
Word ISR	Depend on Number_of_Occurence , a member in document attribute.
And ISR	$\text{Satisfying_Grade} = 0.5^n$, where n is the number of word ISRs
Or ISR	$\text{Satisfying_Grade} = 0.5$
Phrase ISR	We'll be satisfied with one phrase near the title

Heuristics: Word Grading

Global Variable	Description	Value
Anchor_Weight	Relative to body text , how much do we emphasize an occurrence as anchor text ?	★ Highest
Title_Weight	Relative to body text , how much do we emphasize an occurrence as title text ?	High
Position_Weight	Relative to text at bottom , how much do we emphasize an occurrence near the title ?	Medium
Satisfying_Threshold	How many occurrences do we require for a document to be “ satisfying ”?	Low
Slope	How confident we are to abandon “ unsatisfying ” pages?	Low

Heuristics: Phrase Grading

Global Variable	Description	Value
Continue_Weight	If the currently pointed posts of wordISR form a “ phrase ”, how many points can be added?	★ High
Close_Weight	If the currently pointed posts of wordISR are “ close ”, how many points can be added?	Medium
Position_Weight	Relative to ISRs at bottom , how much do we emphasize an occurrence near the title ?	Low
Close_Threshold	What is the maximum distance between two posts of two ISRs to be called “ close ”?	10 words
Satisfying_Threshold	How many phrase/close occurrences do we require for a document to be “ satisfying ”?	1 phrase



Video

